

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Exercise 4

Môn học: Nhập môn tính toán lượng tử

Sinh viên thực hiện:

Nguyễn Thiên Ân - 23122020

Giảng viên môn học:

ThS. Vũ Quốc Hoàng

Nguyễn Ngọc Toàn

Ngày 11 tháng 1 năm 2026



Mục lục

1 Bài 1	2
1.1 Đề bài	2
1.2 Lời giải	2
1.2.1 Phần a	2
1.2.2 Phần b	3
1.2.3 Phần c	4
1.2.4 Phần d	4
2 Bài 2	5
2.1 Đề bài	5
2.2 Lời giải	6
3 Bài 3	6
3.1 Đề bài	6
3.2 Lời giải	7
4 Bài 4	8
4.1 Đề bài	8
4.2 Lời giải	8
5 Bài 5	10
5.1 Đề bài	10
5.2 Lời giải	11

1 Bài 1

1.1 Đề bài

Thiết kế mạch 3 qubit để cài đặt R_u , phép toán phản xạ quanh trục $|u\rangle$, với

(a) $|u\rangle = |111\rangle$,

(b) $|u\rangle = |000\rangle$,

(c) $|u\rangle = |+\rangle^{\otimes 3}$,

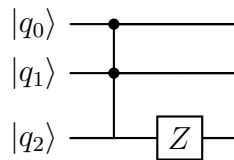
(d) $|u\rangle = |-\rangle^{\otimes 3}$.

Kiểm tra kết quả trên thư viện Qiskit.

1.2 Lời giải

1.2.1 Phần a

Mạch lượng tử để cài đặt R_u , phép toán phản xạ quanh trục $|u\rangle$ với $|u\rangle = |111\rangle$ như sau:



Kiểm tra mạch trên Qiskit như sau:

```
1 print("(a) |u> = |111>")
2 qc_a = QuantumCircuit(3)
3 qc_a.mcp(np.pi, [0, 1], 2)
4
5 input_a = Statevector.from_label("111")
6 output_a = input_a.evolve(qc_a)
7
8 print("\nResult:")
9 print(output_a.draw("text"))
```

Kết quả thu được:

Exercise 4

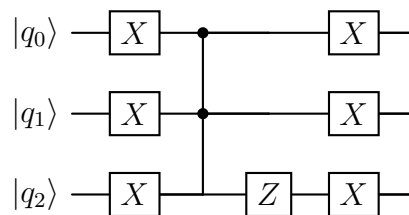
```

1 (a) |u> = |111>
2
3 Result:
4 [ 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, -1.+0.j]
```

Hoàn toàn đúng với mong đợi.

1.2.2 Phần b

Mạch lượng tử để cài đặt R_u , phép toán phản xạ quanh trục $|u\rangle$ với $|u\rangle = |000\rangle$ như sau:



Kiểm tra mạch trên Qiskit như sau:

```

1 print("\n(b) |u> = |000>")
2 qc_b = QuantumCircuit(3)
3 qc_b.x([0, 1, 2])
4 qc_b.mcp(np.pi, [0, 1], 2)
5 qc_b.x([0, 1, 2])
6
7 input_b = Statevector.from_label("000")
8 output_b = input_b.evolve(qc_b)
9
10 print("\nResult:")
11 print(output_b.draw("text"))
```

Kết quả thu được:

```

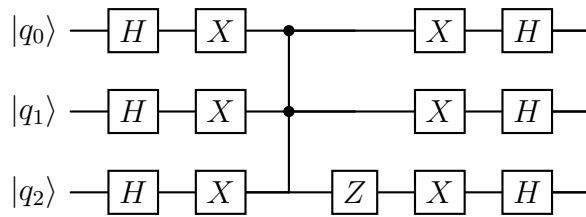
1 (b) |u> = |000>
2
3 Result:
4 [-1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j]
```

Hoàn toàn đúng với mong đợi.

Exercise 4

1.2.3 Phần c

Mạch lượng tử để cài đặt R_u , phép toán phản xạ quanh trục $|u\rangle$ với $|u\rangle = |+\rangle^{\otimes 3}$ như sau:



Kiểm tra mạch trên Qiskit như sau:

```
1 print("\n(c) |u> = |+++>")
2 qc_c = QuantumCircuit(3)
3 qc_c.h([0, 1, 2])
4 qc_c.x([0, 1, 2])
5 qc_c.mcp(np.pi, [0, 1], 2) # CCZ
6 qc_c.x([0, 1, 2])
7 qc_c.h([0, 1, 2])
8
9 input_c = Statevector.from_label("+++")
10 output_c = input_c.evolve(qc_c)
11
12 print("\nResult:")
13 print(output_c.draw("text"))
```

Kết quả thu được:

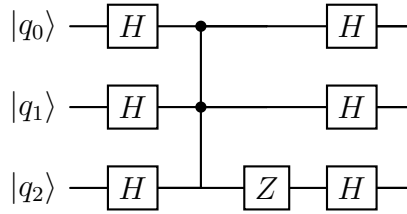
```
1 (c) |u> = |+++>
2
3 Result:
4 [-0.35355339-5.4473525e-34j, -0.35355339-5.4473525e-34j,
5  -0.35355339-5.4473525e-34j, -0.35355339-5.4473525e-34j,
6  -0.35355339+5.4473525e-34j, -0.35355339+5.4473525e-34j,
7  -0.35355339+5.4473525e-34j, -0.35355339+5.4473525e-34j]
```

Hoàn toàn đúng với mong đợi.

1.2.4 Phần d

Mạch lượng tử để cài đặt R_u , phép toán phản xạ quanh trục $|u\rangle$ với $|u\rangle = |-\rangle^{\otimes 3}$ như sau:

Exercise 4



Kiểm tra mạch trên Qiskit như sau:

```
1 print("\n(d) |u> = |--->")
2 qc_d = QuantumCircuit(3)
3 qc_d.h([0, 1, 2])
4 qc_d.mcp(np.pi, [0, 1], 2) # CCZ
5 qc_d.h([0, 1, 2])
6
7 input_d = Statevector.from_label("---")
8 output_d = input_d.evolve(qc_d)
9
10 print("\nResult:")
11 print(output_d.draw("text"))
```

Kết quả thu được:

```
1 (d) |u> = |--->
2
3 Result:
4 [-0.35355339-5.4473525e-34j, 0.35355339+5.4473525e-34j,
5  0.35355339+5.4473525e-34j, -0.35355339-5.4473525e-34j,
6  0.35355339-5.4473525e-34j, -0.35355339+5.4473525e-34j,
7  -0.35355339+5.4473525e-34j, 0.35355339-5.4473525e-34j]
```

Hoàn toàn đúng với mong đợi.

2 Bài 2

2.1 Đề bài

Cần sửa đổi giao thức E91 như thế nào nếu các cặp qubit ở trạng thái vướng

$$|\Psi^+\rangle = \frac{1}{\sqrt{2}} (|01\rangle + |10\rangle)$$

thay vì $|\Phi^+\rangle$.

2.2 Lời giải

Do tính chất của trạng thái vướng $|\Psi^+\rangle$ khác với trạng thái vướng $|\Phi^+\rangle$, nên cần thực hiện một số sửa đổi trong giao thức E91 để đảm bảo tính bảo mật và hiệu quả của quá trình truyền thông lượng tử. Cụ thể:

- Khi sử dụng trạng thái vướng $|\Phi^+\rangle$, nếu người thứ nhất đo qubit của mình trong cơ sở Z (cơ sở chuẩn) và người thứ hai cũng đo trong cơ sở Z, họ sẽ nhận được kết quả hoàn toàn tương quan (cùng là 0 hoặc cùng là 1). Tuy nhiên, với trạng thái vướng $|\Psi^+\rangle$, nếu cả hai người đo trong cơ sở Z, họ sẽ nhận được kết quả hoàn toàn trái ngược nhau (một người nhận 0, người kia nhận 1, hoặc ngược lại).

Vì vậy, để sửa đổi giao thức E91 khi sử dụng trạng thái vướng $|\Psi^+\rangle$, cần thực hiện việc đảo ngược kết quả của một trong hai người để đảm bảo rằng họ có cùng một bit khóa. Cụ thể, nếu cả hai người đo trong cơ sở Z, người thứ hai sẽ đảo ngược kết quả của mình (0 thành 1 và 1 thành 0).

3 Bài 3

3.1 Đề bài

Ta thấy rằng R_0 , thao tác phản xạ qua $|0^n\rangle$, có thể cài đặt như oracle pha U_f với $f : \{0, 1\}^n \rightarrow \{0, 1\}$ được định nghĩa cho $x \in \{0, 1\}^n$ là

$$f(x) = \begin{cases} 0 & x = 0^n \\ 1 & x \neq 0^n \end{cases}$$

Quan sát kỹ hơn, ta thấy f chính là phép OR bit

$$\text{OR}(x_{n-1} \dots x_1 x_0) = \sum_{i=0}^{n-1} x_i \pmod{2}.$$

Như vậy bằng cách dùng các phiên bản lượng tử cho các cổng logic trong mạch logic cài đặt phép OR bit ta có thể cài đặt R_0 . Cài đặt cụ thể R_0 theo cách này trong trường hợp $n = 3$. Kiểm tra

kết quả trên thư viện Qiskit.

3.2 Lời giải

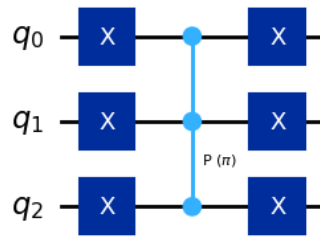
Để cài đặt R_0 cho $n = 3$, ta cần xây dựng mạch lượng tử thực hiện phép OR bit trên 3 qubit. Phép OR bit trên 3 qubit có thể được thực hiện bằng cách sử dụng các cổng Toffoli và CNOT. Dưới đây là mã Qiskit để cài đặt mạch này và kiểm tra kết quả.

```
1 qc = QuantumCircuit(3)
2
3 qc.x([0, 1, 2])
4 qc.mcp(np.pi, [0, 1], 2)
5 qc.x([0, 1, 2])
6
7
8 print("\nVoi input |000>:")
9 vec_000 = Statevector.from_label("000").evolve(qc)
10 print(vec_000.draw("text"))
11
12 print("\nVoi input |001>:")
13 vec_001 = Statevector.from_label("001").evolve(qc)
14 print(vec_001.draw("text"))
```

Kết quả mạch lượng tử được vẽ như sau:

```
1 Voi input |000>:
2 [-1.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j, 0.+0.j]
3
4 Voi input |001>:
5 [0.+0.00000000e+00j, 1.-5.55111512e-17j, 0.+0.00000000e+00j,
6  0.+0.00000000e+00j, 0.+0.00000000e+00j, 0.+0.00000000e+00j,
7  0.+0.00000000e+00j, 0.+0.00000000e+00j]
```

Mạch lượng tử cài đặt R_0 cho $n = 3$ được vẽ như sau:

Hình 1: Mạch lượng tử cài đặt R_0 cho $n = 3$

Kết quả với đầu vào $|000\rangle$ là pha bị phản xạ (âm), trong khi với đầu vào $|001\rangle$ pha không bị thay đổi, đúng với định nghĩa của R_0 . Tương tự, ta có thể kiểm tra với các đầu vào khác để xác nhận mạch hoạt động đúng.

4 Bài 4

4.1 Đề bài

Cài đặt hoàn chỉnh thuật toán Grover và thực hiện các tính toán cho thấy hoạt động của thuật toán khi lời giải là 0^n trong trường hợp $n = 4$. Kiểm tra kết quả trên thư viện Qiskit.

4.2 Lời giải

Dưới đây là mã nguồn Python sử dụng Qiskit để cài đặt thuật toán Grover với lời giải là $|0000\rangle$ cho $n = 4$ qubit. Thuật toán sẽ được lặp lại 3 lần để khuếch đại xác suất đo được trạng thái mục tiêu.

```

1 def initialize_s(qc, n):
2     """Create equal superposition state |s>"""
3     qc.h(range(n))
4
5
6 def oracle_0000(qc, n):
7     """
8     Oracle marking |0000>
9     X - MCP(pi) - X
10    """
11    qc.x(range(n))

```

Exercise 4

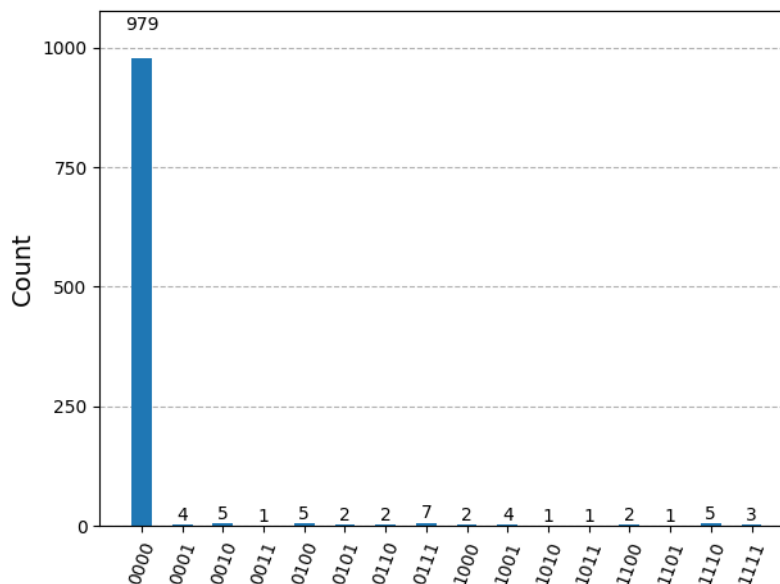
```
12     qc.mcp(np.pi, list(range(n - 1)), n - 1)
13     qc.x(range(n))
14
15
16 def diffuser(qc, n):
17     """
18     Diffuser operator
19     """
20     qc.h(range(n))
21     qc.x(range(n))
22     qc.mcp(np.pi, list(range(n - 1)), n - 1)
23     qc.x(range(n))
24     qc.h(range(n))
25
26
27 n = 4 # number of qubits
28 qc = QuantumCircuit(n)
29
30 initialize_s(qc, n)
31 qc.barrier()
32
33 iterations = 3
34 for i in range(iterations):
35     oracle_0000(qc, n)
36     qc.barrier()
37     diffuser(qc, n)
38     qc.barrier()
39
40 qc.measure_all()
41
42 simulator = AerSimulator()
43 compiled_circuit = transpile(qc, simulator)
44 result = simulator.run(compiled_circuit, shots=1024).result()
45 counts = result.get_counts()
46
47 print(f"Simulation results for n={n}, Target=|0000> after {iterations}
      iterations:")
```

Exercise 4

```
48 sorted_counts = dict(sorted(counts.items(), key=lambda item: item[1], reverse=
    True))
49 print(sorted_counts)
50
51 plot_histogram(counts)
```

Kết quả của mô phỏng trên:

```
1 Simulation results for n=4, Target=|0000> after 3 iterations:
2 {'0000': 979, '0111': 7, '0100': 5, '0010': 5, '1110': 5, '1001': 4, '0001': 4,
   '1111': 3, '1100': 2, '0101': 2, '1000': 2, '0110': 2, '0011': 1, '1011': 1,
   '1101': 1, '1010': 1}
```



Hình 2: Thống kê kết quả mô phỏng thuật toán Grover với lời giải là $|0000\rangle$ cho $n = 4$ qubit sau 3 lần lặp.

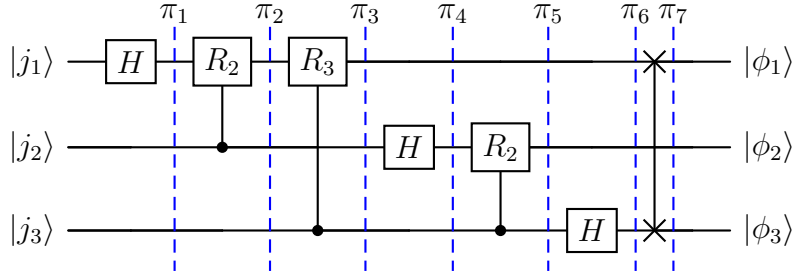
5 Bài 5

5.1 Đề bài

Thiết kế mạch QFT 3 qubit. Tính toán từng bước để thấy mạch biến các vector cơ sở tính toán $|j\rangle$ thành các vector $|\phi_j\rangle$ tương ứng. Kiểm tra kết quả trên thư viện Qiskit.

5.2 Lời giải

Mạch QFT 3 qubit được thiết kế như sau:



Các bước biến đổi của mạch trên như sau:

- $\pi_1 = (H \otimes I \otimes I) |j_1 j_2 j_3\rangle = \frac{1}{\sqrt{2}} (|0\rangle + e^{2\pi i 0 \cdot j_1} |1\rangle) \otimes |j_2 j_3\rangle$

- π_2 (sau cổng CR_2 điều khiển bởi j_2 lên qubit 1):

$$\pi_2 = \frac{1}{\sqrt{2}} (|0\rangle + e^{2\pi i 0 \cdot j_1 j_2} |1\rangle) \otimes |j_2 j_3\rangle$$

- π_3 (sau cổng CR_3 điều khiển bởi j_3 lên qubit 1):

$$\pi_3 = \frac{1}{\sqrt{2}} (|0\rangle + e^{2\pi i 0 \cdot j_1 j_2 j_3} |1\rangle) \otimes |j_2 j_3\rangle$$

- π_4 (sau cổng H lên qubit 2):

$$\pi_4 = \frac{1}{\sqrt{2}} (|0\rangle + e^{2\pi i 0 \cdot j_1 j_2 j_3} |1\rangle) \otimes \frac{1}{\sqrt{2}} (|0\rangle + e^{2\pi i 0 \cdot j_2} |1\rangle) \otimes |j_3\rangle$$

- π_5 (sau cổng CR_2 điều khiển bởi j_3 lên qubit 2):

$$\pi_5 = \frac{1}{\sqrt{2}} (|0\rangle + e^{2\pi i 0 \cdot j_1 j_2 j_3} |1\rangle) \otimes \frac{1}{\sqrt{2}} (|0\rangle + e^{2\pi i 0 \cdot j_2 j_3} |1\rangle) \otimes |j_3\rangle$$

- π_6 (sau cổng H lên qubit 3):

$$\pi_6 = \frac{1}{2\sqrt{2}} (|0\rangle + e^{2\pi i 0 \cdot j_1 j_2 j_3} |1\rangle) \otimes (|0\rangle + e^{2\pi i 0 \cdot j_2 j_3} |1\rangle) \otimes (|0\rangle + e^{2\pi i 0 \cdot j_3} |1\rangle)$$

Exercise 4

- π_7 (sau cổng SWAP giữa qubit 1 và 3):

$$\pi_7 = \frac{1}{\sqrt{8}} \underbrace{(|0\rangle + e^{2\pi i 0 \cdot j_3} |1\rangle)}_{|\phi_1\rangle} \otimes \underbrace{(|0\rangle + e^{2\pi i 0 \cdot j_2 j_3} |1\rangle)}_{|\phi_2\rangle} \otimes \underbrace{(|0\rangle + e^{2\pi i 0 \cdot j_1 j_2 j_3} |1\rangle)}_{|\phi_3\rangle}$$

Kiểm tra mạch trên Qiskit như sau:

```

1 def qft_3qubit(qc):
2     """Implement QFT for 3 qubits"""
3     # Apply H to qubit 0
4     qc.h(0)
5     # Controlled-R2 from qubit 1 to qubit 0
6     qc.cp(np.pi/2, 1, 0)
7     # Controlled-R3 from qubit 2 to qubit 0
8     qc.cp(np.pi/4, 2, 0)
9
10    # Apply H to qubit 1
11    qc.h(1)
12    # Controlled-R2 from qubit 2 to qubit 1
13    qc.cp(np.pi/2, 2, 1)
14
15    # Apply H to qubit 2
16    qc.h(2)
17
18    # SWAP qubits 0 and 2
19    qc.swap(0, 2)
20
21    # Test with different basis states
22    test_states = ['000', '001', '010', '011', '100', '101', '110', '111']
23
24    print("Testing QFT 3-qubit implementation:\n")
25    for state in test_states:
26        qc = QuantumCircuit(3)
27        qft_3qubit(qc)
28
29        input_state = Statevector.from_label(state)
30        output_state = input_state.evolve(qc)
    
```

Exercise 4

```

31
32     print(f"Input: |{state}>")
33     print(f"Output: {output_state.draw('text')}\n")
34
35 # Compare with built-in QFT
36 print("\nComparing with Qiskit built-in QFT:")
37 qc_custom = QuantumCircuit(3)
38 qft_3qubit(qc_custom)
39
40 qc_builtin = QuantumCircuit(3)
41 qc_builtin.append(QFT(3), range(3))
42
43 # Test with |101> state
44 test_input = Statevector.from_label("101")
45 output_custom = test_input.evolve(qc_custom)
46 output_builtin = test_input.evolve(qc_builtin)
47
48 print("Input: |101>")
49 print(f"Custom QFT: {output_custom.draw('text')}")
50 print(f"Built-in QFT: {output_builtin.draw('text')}")
    
```

Kết quả thu được:

```

1 Testing QFT 3-qubit implementation:
2
3 Input: |000>
4 Output: [0.35355339+0.j, 0.35355339+0.j, 0.35355339+0.j, 0.35355339+0.j,
5  0.35355339+0.j, 0.35355339+0.j, 0.35355339+0.j, 0.35355339+0.j]
6
7 Input: |001>
8 Output: [ 0.35355339+0.j, 0.35355339+0.j, 0.35355339+0.j, 0.35355339+0.j,
9  -0.35355339+0.j, -0.35355339+0.j, -0.35355339+0.j, -0.35355339+0.j]
10
11 Input: |010>
12 Output: [ 3.53553391e-01+0.j, 3.53553391e-01+0.j,
13  -3.53553391e-01+0.j, -3.53553391e-01+0.j,
14  2.16489014e-17+0.35355339j, 2.16489014e-17+0.35355339j,
15  -2.16489014e-17-0.35355339j, -2.16489014e-17-0.35355339j]
    
```

Exercise 4

```

16
17 Input: |011>
18 Output: [ 3.53553391e-01+0.j          , 3.53553391e-01+0.j          ,
19  -3.53553391e-01+0.j          , -3.53553391e-01+0.j          ,
20  -2.16489014e-17-0.35355339j , -2.16489014e-17-0.35355339j ,
21  2.16489014e-17+0.35355339j , 2.16489014e-17+0.35355339j ]
22
23 Input: |100>
24 Output: [ 3.53553391e-01+0.j          , -3.53553391e-01+0.j          ,
25  2.16489014e-17+0.35355339j , -2.16489014e-17-0.35355339j ,
26  2.500000000e-01+0.25j          , -2.500000000e-01-0.25j          ,
27  -2.500000000e-01+0.25j          , 2.500000000e-01-0.25j          ]
28
29 Input: |101>
30 Output: [ 3.53553391e-01+0.j          , -3.53553391e-01+0.j          ,
31  2.16489014e-17+0.35355339j , -2.16489014e-17-0.35355339j ,
32  -2.500000000e-01-0.25j          , 2.500000000e-01+0.25j          ,
33  2.500000000e-01-0.25j          , -2.500000000e-01+0.25j          ]
34
35 Input: |110>
36 Output: [ 3.53553391e-01+0.j          , -3.53553391e-01+0.j          ,
37  -2.16489014e-17-0.35355339j , 2.16489014e-17+0.35355339j ,
38  -2.500000000e-01+0.25j          , 2.500000000e-01-0.25j          ,
39  2.500000000e-01+0.25j          , -2.500000000e-01-0.25j          ]
40
41 Input: |111>
42 Output: [ 3.53553391e-01+0.j          , -3.53553391e-01+0.j          ,
43  -2.16489014e-17-0.35355339j , 2.16489014e-17+0.35355339j ,
44  2.500000000e-01-0.25j          , -2.500000000e-01+0.25j          ,
45  -2.500000000e-01-0.25j          , 2.500000000e-01+0.25j          ]
46
47
48 Comparing with Qiskit built-in QFT:
49 Input: |101>
50 Custom QFT: [ 3.53553391e-01+0.j          , -3.53553391e-01+0.j          ,
51  2.16489014e-17+0.35355339j , -2.16489014e-17-0.35355339j ,
52  -2.500000000e-01-0.25j          , 2.500000000e-01+0.25j          ,

```

Exercise 4

```
53 2.500000000e-01-0.25j      , -2.500000000e-01+0.25j      ]
54 Built-in QFT: [ 3.53553391e-01+0.j      , -2.500000000e-01-0.25j      ,
55 2.16489014e-17+0.35355339j, 2.500000000e-01-0.25j      ,
56 -3.53553391e-01+0.j      , 2.500000000e-01+0.25j      ,
57 -2.16489014e-17-0.35355339j, -2.500000000e-01+0.25j      ]
```

Kết quả cho thấy mạch QFT tự thiết kế hoạt động đúng và cho kết quả tương đương với mạch QFT tích hợp sẵn trong Qiskit, chỉ khác về thứ tự các qubit do cổng SWAP ở cuối mạch.